

Assignment: Spring Boot Employee Management REST API

Objective

Develop a Spring Boot Web API application using Java 17 to manage employee information. The application should follow modern best practices, including **layered architecture**, proper **validation**, **Swagger/OpenAPI documentation**, and a **Postman collection** for testing.

Features

Implement CRUD APIs for employee management:

HTTP Method	Operation
GET	Retrieve employee by ID
GET	Retrieve all employees
POST	Create a new employee
PUT	Update a complete employee record
PATCH	Update a specific field (e.g., department)
DELETE	Delete an employee

Additional requirements:

- Create **two endpoints** to upload and fetch images (e.g., employee profile picture)
- Apply **caching** for image-related endpoints
- Create an endpoint that **returns a cookie**
- Use as many relevant **Spring annotations and features** as possible
- You may add extra APIs if needed

Architecture Guidelines

Follow a layered structure:

- **Controller Layer** – Handles API requests and responses
- **Service Layer** – Contains business logic
- **Repository Layer** – Database interaction using Spring Data JPA
- **Entity Layer** – JPA entities representing database tables

Validation & HTTP Status Codes

- Use Bean Validation annotations such as: `@NotBlank`, `@NotNull`, `@Email`, etc.
- Use `@Validated` with validation groups
- Return proper HTTP responses:
 - **200 OK** – Successful fetch, update, or delete
 - **201 Created** – Successful creation
 - **400 Bad Request** – Invalid input
 - **404 Not Found** – Resource not found

API Documentation

- Integrate **Springdoc OpenAPI**
- Generate interactive Swagger UI
- Export OpenAPI documentation as a **JSON file**

Testing

- Create a complete **Postman collection** covering all implemented APIs
- Ensure **all endpoints execute successfully** when the collection is run
- **Save example responses** for each request within the collection

Authentication

- Implement authentication/authorization filter
- Prefer **JWT-based authentication**
- If JWT is not possible, use a **dummy token mechanism**

Additional Requirements

- Implement **global exception handling** for consistent error responses
- Implement **all API versioning strategies** (URI, header, parameter, etc.)

Deliverables

- Spring Boot project following layered architecture
- OpenAPI JSON documentation file (added to repository)
- Postman collection covering all APIs (added to repository)